

Se continuó con el mantenimiento y mejora de la solución automatizada para la generación de reportes de trámites, realizando ajustes al script en Python para optimizar el procesamiento sistemático de la información consolidada y la generación de reportes individuales por municipio, a partir de la plantilla (TuCatastro) previamente definida.

Documentación Código y Reportes:

https://github.com/cientificoacc/2026/tree/main/Reportes_Automatizacion_ACC

Codigo:

```
# LIBRERÍAS
import subprocess, sys

def instalar(paquete):
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q",
paquete])

instalar("python-docx")
instalar("reportlab")
instalar("matplotlib")
subprocess.run(["apt-get", "-qq", "install", "-y", "libreoffice"],
capture_output=True)

# -----
# 1. IMPORTACIONES
# -----

import os
import locale
import pandas as pd
import matplotlib.pyplot as plt
from docx import Document
from docx.shared import Inches, RGBColor
from docx.xml import XmlElement
from docx.xml.ns import qn
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.opc.exceptions import PackageNotFoundError

try:
    locale.setlocale(locale.LC_ALL, 'es_CO.UTF-8')
except Exception:
    try:
        locale.setlocale(locale.LC_ALL, '')
    except Exception:
        pass
```

```

# -----
# 2. CONFIGURACIÓN GENERAL
# -----
BASE_EXCEL      = "/content/Plantilla_Reporte_TuCatastro.xlsx"
PLANTILLA_WORD  = "/content/Mpio_Informe_tramites_Catastral_ACC.docx"
ANIOS           = [2024, 2025]
CARPETA_RAIZ    = "/content/Reportes_ACC_2024_2025"

# Textos estáticos originales de la plantilla que se reemplazarán
# por marcadores la primera vez (función parchear_plantilla)
_TEXTOS_A_MARCADOR = {
    "Esta grafica muestra el mes y año en el que más se han recibido
    radicaciones.":
        "[[ANALISIS_TABLA2]]",
    "El día se la semana que más presenta radicaciones, indica el año en
    que más se han recibido radicaciones.":
        "[[ANALISIS_MES]]",
}
_MARCADOR_POST_IMAGEN2 = "[[ANALISIS_DIA]]"
_MARCADOR_POST_TABLA3  = "[[ANALISIS_TABLA3]]"

# -----
# 3. PARCHEO AUTOMÁTICO DE LA PLANTILLA
# -----
def parchear_plantilla(ruta_plantilla):
    """
    Verifica si la plantilla ya tiene los marcadores nuevos.
    Si no los tiene, reemplaza los textos estáticos por los marcadores
    y agrega los marcadores que faltan después de Imagen 2 y Tabla 3.
    Modifica la plantilla en sitio (sobrescribe el archivo).
    """
    doc = Document(ruta_plantilla)
    paragraphs = doc.paragraphs
    modificado = False

    # a) Reemplazar textos estáticos por marcadores
    for i, p in enumerate(paragraphs):
        texto = p.text.strip()
        for texto_original, marcador in _TEXTOS_A_MARCADOR.items():
            if texto_original in texto and marcador not in texto:
                for run in p.runs:
                    if texto_original in run.text:
                        run.text = run.text.replace(texto_original,
marcador)

```

```

        modificado = True
        break
    else:
        # marcador dividido en runs
        if p.runs:
            p.runs[0].text = marcador
            for run in p.runs[1:]:
                run.text = ""
            modificado = True

# b) Agregar [[ANALISIS_DIA]] después de "Imagen 2. Radicados por día
de la semana"
#     y [[ANALISIS_TABLA3]] después de "Tabla 3. Radicados
resueltos..."
texto_completo = [p.text.strip() for p in paragraphs]

def _marcador_ya_existe(doc, marcador):
    return any(marcador in p.text for p in doc.paragraphs)

def _insertar_marcador_despues(doc, titulo_fragmento, marcador):
    for i, p in enumerate(doc.paragraphs):
        if titulo_fragmento in p.text:
            # Crear nuevo párrafo con el marcador
            nuevo_p = doc.add_paragraph(marcador)
            nuevo_p.alignment = WD_ALIGN_PARAGRAPH.JUSTIFY
            # Moverlo justo después del párrafo del título
            p._element.addnext(nuevo_p._element)
            return True
    return False

if not _marcador_ya_existe(doc, _MARCADOR_POST_IMAGEN2):
    ok = _insertar_marcador_despues(doc, "Imagen 2. Radicados por día
de la semana",
                                   _MARCADOR_POST_IMAGEN2)

    if ok:
        modificado = True

if not _marcador_ya_existe(doc, _MARCADOR_POST_TABLA3):
    ok = _insertar_marcador_despues(doc,
        "Tabla 3. Radicados resueltos desagregados por quien resuelve
y por año de resolución",
        _MARCADOR_POST_TABLA3)

    if ok:
        modificado = True

```

```

    if modificado:
        doc.save(ruta_plantilla)
        print("✅ Plantilla actualizada con los nuevos marcadores.")
    else:
        print("❌ La plantilla ya contiene todos los marcadores. No se realizaron cambios.")

# -----
# 4. NORMALIZACIÓN DE DATOS
# -----
def normalizar_columnas(df):
    df.columns = df.columns.str.strip()
    df.columns = (
        df.columns
        .str.normalize('NFKD')
        .str.encode('ascii', errors='ignore')
        .str.decode('utf-8')
    )
    return df

def estandarizar_estado(df, col_estado="Estado"):
    if col_estado not in df.columns:
        print(f"⚠️ Columna '{col_estado}' no encontrada.")
        return df
    mapeo = {
        "finalizado_por_desistimiento": "Finalizado por desistimiento",
        "generacion_acto_administrativo": "Generacion de acto
administrativo",
        "informado_notificado_cumplido": "Informado notificado
cumplido",
        "no_procedente": "No procedente",
        "radicado": "Radicado",
    }
    df = df.copy()
    s_low = df[col_estado].astype(str).str.strip().str.lower()
    df[col_estado] = s_low.map(mapeo).fillna(df[col_estado])
    return df

def filtrar_tipos_excluidos(df, columna_tipo="Tipo",
excluidos=("Certificado", "Carta")):
    if columna_tipo not in df.columns:
        print(f"⚠️ Columna '{columna_tipo}' no encontrada.")
        return df
    patron = "|".join(str(x) for x in excluidos)

```

```

    mask = df[columna_tipo].astype(str).str.contains(patron, case=False,
na=False)
    return df[~mask].copy()

def pct(numerador, denominador):
    if not denominador or pd.isna(denominador):
        return 0.0
    return (numerador / denominador) * 100.0

def detectar_columna_fecha(df):
    posibles = [c for c in df.columns if 'fecha' in c.lower() and 'rad' in
c.lower()]
    if not posibles:
        raise ValueError("✗ No se encontró columna de fecha de
radicación.")
    return posibles[0]

def obtener_cod_mun(df_mun):
    if "COD_MUN" not in df_mun.columns:
        return "SIN_COD"
    vals = df_mun["COD_MUN"].dropna().astype(str).unique().tolist()
    if not vals:
        return "SIN_COD"
    cod = vals[0]
    return cod[:-2].strip() if cod.endswith(".0") else cod.strip()

# -----
# 5. CONSTRUCCIÓN DE TABLAS
# -----

def construir_tabla1(df):
    t = (
        df.groupby(['Estado', 'RAD_ANO'])['Numero radicado']
        .nunique()
        .unstack(fill_value=0)
        .reindex(columns=[2024, 2025], fill_value=0)
    )
    t['Total general'] = t.sum(axis=1)
    t.loc['Total general'] = t.sum()
    return (
        t.reset_index()
        .rename(columns={'Estado': 'Estado de la radicación', 2024:
'2024', 2025: '2025'})
    )

def construir_tabla2(df):

```

```

t = (
    df.groupby(['Tipo', 'RAD_ANO'])['Numero radicado']
        .nunique()
        .unstack(fill_value=0)
        .reindex(columns=[2024, 2025], fill_value=0)
)
t['Total general'] = t.sum(axis=1)
t = t.reset_index()
t.columns = ['Tipo de trámite', '2024', '2025', 'Total general']
total_row = pd.DataFrame([
    'Tipo de trámite': 'Total general',
    '2024': t['2024'].sum(), '2025': t['2025'].sum(),
    'Total general': t['Total general'].sum()
])
return pd.concat([t, total_row], ignore_index=True)

def construir_tabla3(df):
    df = df.copy()
    df['RES_ANO_GROUP'] = 'En proceso'
    df.loc[df['RES_ANO'] == 2024, 'RES_ANO_GROUP'] = 'Resolución 2024'
    df.loc[df['RES_ANO'] == 2025, 'RES_ANO_GROUP'] = 'Resolución 2025'
    t = (
        df.groupby(['RAD_ANO', 'Tipo', 'RES_ANO_GROUP'])['Numero
radicado']
            .nunique()
            .unstack(fill_value=0)
            .reindex(columns=['En proceso', 'Resolución 2024', 'Resolución
2025'], fill_value=0)
    )
    t['Total general'] = t.sum(axis=1)
    t = t.reset_index()
    t.columns = ['Año', 'Tipo de trámite', 'En proceso',
                'Resolución 2024', 'Resolución 2025', 'Total general']
    total_row = pd.DataFrame([
        'Año': 'Total general', 'Tipo de trámite': '',
        'En proceso': t['En proceso'].sum(),
        'Resolución 2024': t['Resolución 2024'].sum(),
        'Resolución 2025': t['Resolución 2025'].sum(),
        'Total general': t['Total general'].sum()
    ])
    return pd.concat([t, total_row], ignore_index=True)

# -----
# 6. FUNCIONES DE ANÁLISIS DESCRIPTIVO
# -----

```

```

def construir_textos_analisis(df_mun, mun, años=(2024, 2025)):
    """Textos para [[DESC_RADICADOS]] y [[ANALISIS_TABLA1]]."""
    total = int(df_mun['Numero radicado'].nunique())
    finalizados = int(df_mun[df_mun['Estado'].eq("Informado notificado
cumplido")])

    ['Numero radicado'].nunique())
    pct_fin = pct(finalizados, total)
    territorio = int(df_mun[~df_mun['OFICINA DE GESTION'].astype(str)
                        .str.upper().str.contains("CENTRAL",
na=False)])

    ['Numero radicado'].nunique())
    pct_terr = pct(territorio, total)

    texto_pre = (
        f"La siguiente tabla corresponde al estado de los radicados por
año y con un total general. "
        f"En {mun} se presenta un total de {total:,} radicados entre los
años {años[0]} y {años[1]}. "
        f"Los valores se desagregan para los años {años[0]} y {años[1]}.
"
        f"Para estos dos años se han finalizado el {pct_fin:.0f}% de los
trámites recibidos."
    )
    texto_post = (
        f"De los {total:,} radicados del municipio de {mun}, el
{pct_terr:.1f}% "
        f"representan trámites que el enlace territorial de la ACC
resuelve en el municipio "
        f"y el porcentaje restante se resuelve a nivel central."
    )
    return texto_pre, texto_post

def construir_analisis_tabla2(df_mun, mun):
    """Texto para [[ANALISIS_TABLA2]] – después de la Tabla 2."""
    tabla2 = construir_tabla2(df_mun)
    datos = tabla2[tabla2['Tipo de trámite'] != 'Total general'].copy()

    if datos.empty:
        return "No se registraron trámites por tipo en el periodo
analizado."

    datos = datos.sort_values('Total general', ascending=False)
    fila_total = tabla2[tabla2['Tipo de trámite'] == 'Total general']

```

```

    total_gral    = int(fila_total['Total general'].values[0]) if not
fila_total.empty else 1
    total_2024    = int(fila_total['2024'].values[0]) if not
fila_total.empty else 0
    total_2025    = int(fila_total['2025'].values[0]) if not
fila_total.empty else 0

    tipo1        = datos.iloc[0]['Tipo de trámite']
    cnt1         = int(datos.iloc[0]['Total general'])
    pct1         = pct(cnt1, total_gral)

    top2 = ""
    if len(datos) >= 2:
        tipo2 = datos.iloc[1]['Tipo de trámite']
        cnt2  = int(datos.iloc[1]['Total general'])
        pct2  = pct(cnt2, total_gral)
        top2  = (f"En segundo lugar se ubica '{tipo2}' con {cnt2:},
radicados "
                f"({pct2:.1f}% del total). ")

    if total_2024 > 0 and total_2025 > 0:
        diff      = total_2025 - total_2024
        pct_var   = pct(abs(diff), total_2024)
        tendencia = (
            f"Entre 2024 ({total_2024:}, radicados) y 2025 ({total_2025:},
radicados) se registra "
            f"un {'incremento' if diff > 0 else 'decremento'} del
{pct_var:.1f}%."
        ) if diff != 0 else (
            f"El volumen de radicados se mantuvo estable entre 2024 y 2025
"
            f"({total_2024:}, radicados por año).")
        )
    elif total_2024 == 0:
        tendencia = f"Los radicados corresponden únicamente al año 2025
({total_2025:}, trámites). "
    else:
        tendencia = f"Los radicados corresponden únicamente al año 2024
({total_2024:}, trámites). "

    return (
        f"De acuerdo con la tabla anterior, el tipo de trámite con mayor
participación en {mun} "
        f"es '{tipo1}', con {cnt1:}, radicados, equivalente al {pct1:.1f}%
del total del periodo. "
    )

```

```

        f"{top2}{tendencia}"
    )

def construir_analisis_grafica_mes(df_mun, mun):
    """Texto para [[ANALISIS_MES]] - después de la Imagen 1."""
    try:
        col_fecha = detectar_columna_fecha(df_mun)
    except ValueError:
        return "No fue posible analizar la distribución mensual."

    df = df_mun.copy()
    df[col_fecha] = pd.to_datetime(df[col_fecha], errors='coerce')
    df['MES_NUM'] = df[col_fecha].dt.month

    meses_es = {
        1: 'enero', 2: 'febrero', 3: 'marzo', 4: 'abril', 5: 'mayo', 6: 'junio',
        7: 'julio', 8: 'agosto', 9: 'septiembre', 10: 'octubre',
        11: 'noviembre', 12: 'diciembre'
    }

    trimestres = {
        1: 'primer trimestre (enero-marzo)',
        2: 'segundo trimestre (abril-junio)',
        3: 'tercer trimestre (julio-septiembre)',
        4: 'cuarto trimestre (octubre-diciembre)'
    }

    t = (
        df.groupby(['MES_NUM', 'RAD_ANO'])['Numero radicado']
        .nunique()
        .unstack(fill_value=0)
        .reindex(columns=[2024, 2025], fill_value=0)
        .reindex(range(1, 13), fill_value=0)
    )
    t['total'] = t[2024] + t[2025]

    mes_pico_num = int(t['total'].idxmax())
    mes_pico = meses_es[mes_pico_num]
    vol_pico = int(t.loc[mes_pico_num, 'total'])

    picos_año = []
    for anio in [2024, 2025]:
        if t[anio].sum() > 0:
            m = int(t[anio].idxmax())
            picos_año.append(

```

```

        f"en {anio} fue {meses_es[m]} con {int(t.loc[m, anio]):,}
radicados"
    )
    texto_picos = "; ".join(picos_año) + "." if picos_año else ""

    t['trim'] = ((t.index - 1) // 3) + 1
    trim_pico =
trimestres.get(int(t.groupby('trim')['total'].sum().idxmax()), "")

    return (
        f"La gráfica anterior muestra la distribución mensual de los
radicados en {mun} "
        f"para los años 2024 y 2025. El mes con mayor volumen acumulado en
el periodo es "
        f"{mes_pico}, concentrando {vol_pico:,} radicados. Desagregado por
año: {texto_picos} "
        f"En términos trimestrales, el {trim_pico} es el de mayor
actividad, lo cual puede "
        f"asociarse con los ciclos de gestión predial del municipio y los
periodos de mayor "
        f"demanda ciudadana ante la ventanilla catastral."
    )

def construir_analisis_grafica_dia(df_mun, mun):
    """Texto para [[ANALISIS_DIA]] – después de la Imagen 2."""
    try:
        col_fecha = detectar_columna_fecha(df_mun)
    except ValueError:
        return "No fue posible analizar la distribución por día de
semana."

    df = df_mun.copy()
    df[col_fecha] = pd.to_datetime(df[col_fecha], errors='coerce')
    df['DIA_NUM'] = df[col_fecha].dt.weekday

    dias_es = {
        0: 'lunes', 1: 'martes', 2: 'miércoles',
        3: 'jueves', 4: 'viernes', 5: 'sábado', 6: 'domingo'
    }
    t = (
        df.groupby(['DIA_NUM', 'RAD_ANO'])['Numero radicado']
        .nunique()
        .unstack(fill_value=0)
        .reindex(columns=[2024, 2025], fill_value=0)
    )

```

```

        .reindex(range(0, 7), fill_value=0)
    )
    t['total'] = t[2024] + t[2025]

    dia_pico_num      = int(t['total'].idxmax())
    dia_pico          = dias_es[dia_pico_num]
    vol_pico          = int(t.loc[dia_pico_num, 'total'])
    dia_min_lab_num   = int(t.loc[0:4, 'total'].idxmin())
    dia_min_lab       = dias_es[dia_min_lab_num]
    vol_min           = int(t.loc[dia_min_lab_num, 'total'])

    total_semana      = int(t.loc[0:4, 'total'].sum())
    total_finde       = int(t.loc[5:6, 'total'].sum())
    total_global      = total_semana + total_finde
    pct_sem           = pct(total_semana, total_global)

    texto_finde = (
        f"El {pct_sem:.1f}% de las radicaciones corresponde a días hábiles  

(lunes a viernes) "
        f"y el {100 - pct_sem:.1f}% al fin de semana."
        if total_finde > 0
        else "La totalidad de los radicados fue registrada en días hábiles  

(lunes a viernes). "
    )

    return (
        f"Respecto a la distribución por día de la semana, el {dia_pico}  

concentra el mayor "
        f"volumen de radicaciones en {mun} con {vol_pico:,} trámites en el  

periodo analizado, "
        f"mientras que el {dia_min_lab} es el día hábil de menor actividad  

({vol_min:,} radicados). "
        f"{texto_finde} Este comportamiento puede orientar la planeación  

del recurso humano y la "
        f"disponibilidad de atención al usuario en la ventanilla  

catastral."
    )

def construir_analisis_tabla3(df_mun, mun):
    """Texto para [[ANALISIS_TABLA3]] – después de la Tabla 3."""
    tabla3      = construir_tabla3(df_mun)
    fila_total  = tabla3[tabla3['Año'] == 'Total general']

    if fila_total.empty:

```

```

        return "No se encontraron datos de resolución para el municipio."

    en_proceso      = int(filas_total['En proceso'].values[0])
    res_2024        = int(filas_total['Resolución 2024'].values[0])
    res_2025        = int(filas_total['Resolución 2025'].values[0])
    total           = int(filas_total['Total general'].values[0])

    if total == 0:
        return "No se registraron radicados en el periodo analizado."

    total_res       = res_2024 + res_2025
    pct_res         = pct(total_res, total)
    pct_proc        = pct(en_proceso, total)
    pct_r24         = pct(res_2024, total_res) if total_res > 0 else 0
    pct_r25         = pct(res_2025, total_res) if total_res > 0 else 0

    if res_2024 > 0 and res_2025 > 0:
        dist = (
            f"De los trámites resueltos, el {pct_r24:.1f}% obtuvo "
            f"resolución durante 2024 "
            f"y el {pct_r25:.1f}% durante 2025. "
        )
    elif res_2024 > 0:
        dist = "La totalidad de los trámites resueltos obtuvo respuesta en "
    elif res_2025 > 0:
        dist = "La totalidad de los trámites resueltos obtuvo respuesta en "
    else:
        dist = ""

    if pct_proc > 50:
        carga = (
            f"El alto porcentaje de trámites aún en proceso "
            f"({pct_proc:.1f}%) indica una carga "
            f"operativa significativa que requiere atención prioritaria en "
            f"la vigencia en curso."
        )
    elif pct_proc > 20:
        carga = (
            f"El {pct_proc:.1f}% de los trámites permanece en proceso, "
            f"representando la carga "
            f"pendiente de gestión para la ACC en {mun}."
        )
    else:

```

```

        carga = (
            f"El municipio presenta un alto nivel de resolución:
            únicamente el {pct_proc:.1f}% "
            f"de los trámites permanece en proceso al corte del análisis."
        )

    return (
        f"La tabla anterior permite evidenciar la capacidad de resolución
        de trámites en {mun}. "
        f"Del total de {total:}, radicados en el periodo 2024-2025, el
        {pct_res:.1f}% "
        f"({total_res:}, trámites) han sido resueltos. {dist}{carga}"
    )

# -----
# 7. REEMPLAZO DE MARCADORES EN PLANTILLA
# -----
def reemplazar_marcador(doc, marcador, texto_nuevo):
    """
    Reemplaza un marcador en la plantilla preservando el estilo del
    párrafo.
    Aplica alineación justificada y elimina cursiva del texto reemplazado.
    Retorna True si encontró y reemplazó el marcador.
    """
    for p in doc.paragraphs:
        if marcador not in p.text:
            continue

        # Caso simple: el marcador está dentro de un solo run
        for run in p.runs:
            if marcador in run.text:
                run.text = run.text.replace(marcador, texto_nuevo)
                run.font.italic = False
                p.alignment = WD_ALIGN_PARAGRAPH.JUSTIFY
                return True

        # Caso compuesto: el marcador está partido entre varios runs
        full = "".join(r.text for r in p.runs)
        if marcador in full:
            # Vaciar todos los runs y escribir en el primero
            nuevo_texto = full.replace(marcador, texto_nuevo)
            if p.runs:
                p.runs[0].text = nuevo_texto
                p.runs[0].font.italic = False
                for run in p.runs[1:]:

```

```

        run.text = ""
    else:
        new_run = p.add_run(nuevo_texto)
        new_run.font.italic = False
    p.alignment = WD_ALIGN_PARAGRAPH.JUSTIFY
    return True

return False

# -----
# 8. FORMATO DE TABLAS WORD
# -----
def formatear_tabla_word(tabla):
    AZUL = "2F5496"
    for row in tabla.rows:
        for cell in row.cells:
            for par in cell.paragraphs:
                par.alignment = WD_ALIGN_PARAGRAPH.CENTER

    # Encabezado
    for cell in tabla.rows[0].cells:
        tc = cell._tc
        tcPr = tc.get_or_add_tcPr()
        for child in list(tcPr):
            if child.tag == qn('w:shd'):
                tcPr.remove(child)
        shd = XmlElement('w:shd')
        shd.set(qn('w:fill'), AZUL)
        tcPr.append(shd)
        for par in cell.paragraphs:
            for run in par.runs:
                run.font.bold = True
                run.font.color.rgb = RGBColor(255, 255, 255)

    # Filas de totales
    for row in tabla.rows[1:]:
        if "TOTAL GENERAL" in row.cells[0].text.strip().upper():
            for cell in row.cells:
                tc = cell._tc
                tcPr = tc.get_or_add_tcPr()
                for child in list(tcPr):
                    if child.tag == qn('w:shd'):
                        tcPr.remove(child)
                shd = XmlElement('w:shd')
                shd.set(qn('w:fill'), "FFFFFF")

```

```

        tcPr.append(shd)
    for par in cell.paragraphs:
        for run in par.runs:
            run.font.bold = True
            run.font.color.rgb = RGBColor(0, 0, 0)

# -----
# 9. INSERCIÓN DE TABLAS E IMÁGENES (sin análisis - ya usa marcadores)
# -----
def reemplazar_bloque(doc, titulo_busqueda, tipo, df=None,
ruta_imagen=None):
    """
    Busca el título en la plantilla e inserta la tabla o imagen
    justo debajo de él. El análisis se maneja por separado mediante
    los marcadores [...] que ya están en el documento.
    """
    paragraphs = doc.paragraphs
    inicio = None

    for i, p in enumerate(paragraphs):
        if titulo_busqueda.strip() in p.text.strip():
            inicio = i
            break

    if inicio is None:
        print(f"⚠ Título no encontrado: '{titulo_busqueda}'")
        return

    titulo_parrafo = paragraphs[inicio]

    if tipo == "tabla":
        tabla = doc.add_table(rows=df.shape[0] + 1, cols=df.shape[1])
        tabla.style = "Table Grid"
        for ci, col in enumerate(df.columns):
            tabla.rows[0].cells[ci].text = str(col)
        for ri in range(df.shape[0]):
            for ci in range(df.shape[1]):
                tabla.rows[ri + 1].cells[ci].text = str(df.iloc[ri, ci])
        formatear_tabla_word(tabla)
        titulo_parrafo._element.addnext(tabla._element)

    elif tipo == "imagen":
        nuevo_p = doc.add_paragraph()
        nuevo_p.alignment = WD_ALIGN_PARAGRAPH.CENTER
        nuevo_p.add_run().add_picture(ruta_imagen, width=Inches(4.5))

```

```

        titulo_parrafo._element.addnext(nuevo_p._element)

# -----
# 10. FUNCIONES DE GRÁFICAS
# -----
def graficar_mes(df, ruta):
    col_fecha = detectar_columna_fecha(df)
    df = df.copy()
    df[col_fecha] = pd.to_datetime(df[col_fecha], errors='coerce')
    df['MES_NUM'] = df[col_fecha].dt.month

    meses_es = {
        1:'Enero', 2:'Febrero', 3:'Marzo', 4:'Abril', 5:'Mayo', 6:'Junio',
        7:'Julio', 8:'Agosto', 9:'Septiembre', 10:'Octubre',
        11:'Noviembre', 12:'Diciembre'
    }

    t = (
        df.groupby(['MES_NUM', 'RAD_ANO'])['Numero radicado']
        .nunique()
        .unstack(fill_value=0)
        .reindex(columns=[2024, 2025], fill_value=0)
        .reindex(range(1, 13), fill_value=0)
    )

    t.index = t.index.map(meses_es)
    ax = t.plot(kind='bar')
    plt.xlabel("Mes")
    plt.ylabel("Cantidad")
    plt.legend(title="Año")
    plt.xticks(rotation=45)
    for container in ax.containers:
        ax.bar_label(container, fontsize=8)
    max_val = t.values.max()
    plt.ylim(0, max_val * 1.15 if max_val > 0 else 1)
    plt.tight_layout()
    plt.savefig(ruta)
    plt.close()

def graficar_dia(df, ruta):
    col_fecha = detectar_columna_fecha(df)
    df = df.copy()
    df[col_fecha] = pd.to_datetime(df[col_fecha], errors='coerce')
    df['DIA_NUM'] = df[col_fecha].dt.weekday

    dias_es = {
        0:'Lunes', 1:'Martes', 2:'Miércoles', 3:'Jueves',
        4:'Viernes', 5:'Sábado', 6:'Domingo'
    }

```

```

}
df['DIA'] = df['DIA_NUM'].map(dias_es)
t = (
    df.groupby(['DIA_NUM', 'DIA', 'RAD_ANO'])['Numero radicado']
      .nunique()
      .unstack(fill_value=0)
      .reindex(columns=[2024, 2025], fill_value=0)
      .reset_index()
      .sort_values('DIA_NUM')
      .set_index('DIA')[[2024, 2025]]
)
ax = t.plot(kind='bar')
plt.xlabel("Día")
plt.ylabel("Cantidad")
plt.legend(title="Año")
plt.xticks(rotation=45)
for container in ax.containers:
    ax.bar_label(container, fontsize=8)
plt.ylim(0, t.values.max() * 1.15 if t.values.max() > 0 else 1)
plt.tight_layout()
plt.savefig(ruta)
plt.close()

# -----
# 11. CONVERSION DOCX → PDF
# -----
def convertir_a_pdf(docx_path, carpeta_pdf):
    os.makedirs(carpeta_pdf, exist_ok=True)
    subprocess.run([
        "soffice", "--headless", "--nologo", "--nofirststartwizard",
        "--convert-to", "pdf", "--outdir",
        os.path.abspath(carpeta_pdf), os.path.abspath(docx_path)
    ], check=True)

# -----
# 12. GENERADOR PRINCIPAL DE REPORTE
# -----
def generar_sistema_reportes(municipio=None):

    # — Carga y limpieza —————
    df = pd.read_excel(BASE_EXCEL, sheet_name='CRUDOS')
    df = normalizar_columnas(df)
    df = df[df['RAD_ANO'].isin(ANIOS)]
    df = filtrar_tipos_excluidos(df, columna_tipo="Tipo",
excluidos=("Certificado", "Carta"))

```

```

df = estandarizar_estado(df, col_estado="Estado")

municipios = [municipio] if municipio else df['NOM_MUN'].unique()
os.makedirs(CARPETA_RAIZ, exist_ok=True)

for mun in municipios:
    df_mun = df[df['NOM_MUN'] == mun].copy()
    if df_mun.empty:
        print(f"⚠ Sin datos para {mun}. Se omite.")
        continue

    # — Carpetas —————
    cod_mun = obtener_cod_mun(df_mun)
    base_mun = f"{CARPETA_RAIZ}/{cod_mun}_{mun}"
    os.makedirs(f"{base_mun}/01_Word", exist_ok=True)
    os.makedirs(f"{base_mun}/02_Graficas", exist_ok=True)
    os.makedirs(f"{base_mun}/03_Pdf", exist_ok=True)

    # — Tablas —————
    tabla1 = construir_tabla1(df_mun)
    tabla2 = construir_tabla2(df_mun)
    tabla3 = construir_tabla3(df_mun)

    # — Gráficas —————
    ruta_mes = f"{base_mun}/02_Graficas/Radicados_por_mes.png"
    ruta_dia = f"{base_mun}/02_Graficas/Radicados_por_dia.png"
    graficar_mes(df_mun, ruta_mes)
    graficar_dia(df_mun, ruta_dia)

    # — Textos de análisis —————
    mun_up = mun.upper()
    texto_pre_t1, texto_post_t1 =
construir_textos_analisis(df_mun, mun_up, tuple(ANIOS))
    analisis_t2 =
construir_analisis_tabla2(df_mun, mun_up)
    analisis_mes =
construir_analisis_grafica_mes(df_mun, mun_up)
    analisis_dia =
construir_analisis_grafica_dia(df_mun, mun_up)
    analisis_t3 =
construir_analisis_tabla3(df_mun, mun_up)

    # — Cargar plantilla —————
    try:
        doc = Document(PLANTILLA_WORD)

```

```

except (PackageNotFoundError, FileNotFoundError):
    print("⚠ Plantilla no encontrada. Se generará documento
nuevo.")

    doc = Document()

# — Reemplazar nombre del municipio —————
for p in doc.paragraphs:
    for run in p.runs:
        if "MPIO" in run.text:
            run.text = run.text.replace("MPIO", mun_up)

# — Reemplazar TODOS los marcadores —————
# (en el orden en que aparecen en el documento)
marcadores = {
    "[[DESC_RADICADOS]]": texto_pre_t1,
    "[[ANALISIS_TABLA1]]": texto_post_t1,
    "[[ANALISIS_TABLA2]]": analisis_t2,
    "[[ANALISIS_MES]]": analisis_mes,
    "[[ANALISIS_DIA]]": analisis_dia,
    "[[ANALISIS_TABLA3]]": analisis_t3,
}
for marcador, texto in marcadores.items():
    encontrado = reemplazar_marcador(doc, marcador, texto)
    if not encontrado:
        print(f" ⚠ Marcador '{marcador}' no encontrado en la
plantilla.")

# — Insertar tablas e imágenes —————
# addnext coloca cada elemento justo debajo del título;
# los marcadores [[ANALISIS_...]] ya fueron reemplazados
# por el texto y quedan en su posición natural.
reemplazar_bloque(
    doc,
    "Clasificación de los radicados para trámites de conservación
catastral",
    tipo="tabla", df=tabla1
)
reemplazar_bloque(
    doc,
    "Radicados por tipo y quien resuelve",
    tipo="tabla", df=tabla2
)
reemplazar_bloque(
    doc, "Radicados por mes",
    tipo="imagen", ruta_imagen=ruta_mes

```

```

    )
    reemplazar_bloque(
        doc, "Radicados por día de la semana",
        tipo="imagen", ruta_imagen=ruta_dia
    )
    reemplazar_bloque(
        doc,
        "Radicados resueltos desagregados por quien resuelve y por año
de resolución",
        tipo="tabla", df=tabla3
    )

    # — Guardar Word —————
    word_path =
f"{base_mun}/01_Word/{mun}_Informe_Final_ACC_2024_2025.docx"
    doc.save(word_path)
    print(f"✅ Word generado: {mun}")

    # — Exportar PDF —————
    try:
        convertir_a_pdf(word_path, f"{base_mun}/03_Pdf")
        print(f"📄 PDF generado: {mun}")
    except Exception as e:
        print(f"⚠️ PDF no generado para {mun}: {e}")

# =====
# 13. EJECUCIÓN
# =====

# Paso 1 - Parchar la plantilla (solo la primera vez o si falta algún
marcador)
#parchear_plantilla(PLANTILLA_WORD)

# Paso 2 - Generar reportes
#generar_sistema_reportes("GUATAVITA")    # ← municipio específico

# Para todos los municipios descomenta la siguiente línea:
generar_sistema_reportes()

```